Cahier des charges

Mini-projets

MLOps : stratégies de déploiement et considérations éthiques

ENSAF/ISDIA3

Mohammed AIRAJ

Objectif des mini-projets

Concevoir, évaluer et industrialiser un système fondé sur l'apprentissage automatique ou l'IA générative, en mettant en pratique la reproductibilité, le suivi des expérimentations, les tests, le CI/CD, le déploiement, le monitoring et la gouvernance.

Année universitaire 2026–2027

Contents

1 Présentation **2**
1.1 Lien avec les chapitres du cours . . . 2

2 Organisation et règles communes **2**
2.1 Socle technique commun . . . 2
2.2 Exigences minimales communes . . . 3

3 Projet 1 – De l'étude de cas à l'industrialisation d'un assistant RAG **3**
3.1 Contexte et objectif . . . 3
3.2 Choix du domaine . . . 4
3.3 A réaliser . . . 4
3.4 Architecture minimale attendue . . . 5
3.5 Évaluation et indicateurs . . . 5
3.6 Livrables spécifiques . . . 5

4 Projet 2 – RAG multi-sources : reproductibilité, évaluation et sécurité documentaire **6**
4.1 Contexte et objectif . . . 6
4.2 Architecture imposée pour le périmètre obligatoire . . . 6
4.3 A réaliser . . . 6
4.4 Sécurité documentaire . . . 6
4.5 Livrables spécifiques . . . 6

5 Projet 3 – Agent intelligent : orchestration d'outils, observabilité et gouvernance **7**
5.1 Contexte et objectif . . . 7
5.2 Périmètre fonctionnel . . . 7
5.3 Travail demandé . . . 7
5.4 Traçabilité et observabilité . . . 8
5.5 Évaluation et sécurité . . . 8
5.6 Livrables spécifiques . . . 8

6 Projet 4 - Industrialisation d'un pipeline d'apprentissage automatique classique **8**
6.1 Contexte et objectif . . . 8
6.2 Domaines possibles . . . 9
6.3 Travail demandé . . . 9
6.4 Évaluation et gouvernance . . . 9
6.5 Livrables spécifiques . . . 9

7 Extensions facultatives **10**

8 Socle de qualité : tests et quality gates **10**

9 Monitoring minimal attendu **11**

10 Livrables communs **11**

11 Conseils Pratiques **11**

1. Présentation

Les mini-projets ont pour objectif de mobiliser de manière intégrée les notions abordées dans les chapitres du cours. Il ne s'agit pas uniquement de développer une application qui produit une prédiction ou une réponse, mais de démontrer qu'un système d'IA peut être construit, évalué, versionné, testé, déployé et surveillé dans des conditions maîtrisées.

Quatre sujets sont proposés. Ils couvrent des situations complémentaires :

- un assistant documentaire fondé sur un pipeline RAG ;
- un système RAG multi-sources centré sur la reproductibilité et la sécurité documentaire ;
- un agent utilisant des outils, avec traçabilité opérationnelle et gouvernance ;
- un pipeline d'apprentissage automatique classique industrialisé.

Vous devez consacrer l'essentiel de vos efforts à l'ingénierie MLOps : données et artefacts versionnés, expérimentation, évaluation, tests, quality gates, automatisation, déploiement, observabilité et gouvernance.

1.1 Lien avec les chapitres du cours

Chapitre	Compétences mobilisées dans les mini-projets
Chapitre 1 – Introduction au MLOps et aux systèmes d'IA générative	Comprendre le cycle de vie, distinguer MLOps, LLMOps et GenAIOps, identifier les composants d'un système et les besoins d'observabilité.
Chapitre 2 – Scénario pratique : chatbot support client	Analyser un besoin métier, définir des indicateurs de succès et raisonner sur les compromis entre qualité, latence, coût et satisfaction. Ce chapitre est une étude de cas conceptuelle.
Chapitre 3 – Reproductibilité, versionnement et suivi des expériences	Versionner le code, les données et les artefacts ; suivre les expériences ; documenter les configurations et pouvoir reconstruire un résultat.
Chapitre 4 – RAG	Construire un pipeline RAG monolithique : ingestion, préparation, embeddings, index vectoriel, retrieval et génération ; évaluer la récupération et les réponses.
Chapitre 5 – CI/CD, tests et gouvernance des modèles	Automatiser les tests, définir des quality gates, gérer les versions, organiser la promotion d'une version et documenter les contrôles de sécurité et de gouvernance.

2. Organisation et règles communes

2.1 Socle technique commun

Les outils ci-dessous constituent une proposition. Des alternatives peuvent être proposées, à condition d'être justifiées.

Besoin	Outils ou solutions possibles
Gestion du code	Git et dépôt distant (GitHub).
Versionnement des données et artefacts	DVC, Git LFS ou mécanisme équivalent adapté à la taille des fichiers.
Suivi des expérimentations	MLflow.
Modèles et exécution locale	Scikit-learn pour le ML classique ; Ollama et un modèle local quantifié ou une API autorisée pour les projets GenAI (Groq par exemple comme vu en TP).
Conteneurisation	Docker (optionnel).
Automatisation	GitHub Actions
Tests	pytest et tests adaptés aux composants du projet.
Évaluation et monitoring	Scripts de mesure, journaux structurés et tableau de bord simple.

2.2 Exigences minimales communes

Vous devez fournir :

1. un dépôt Git organisé (plus simple dans GitHub), avec un README permettant de comprendre et d'exécuter le projet ;
2. une procédure de reproduction précisant les versions, configurations, commandes et prérequis ;
3. un versionnement explicite des données ou documents pertinents, des configurations et des artefacts importants ;
4. un suivi des expérimentations avec les paramètres, métriques et versions associées ;
5. des tests automatisés adaptés au système ;
6. une chaîne CI exécutant au minimum les contrôles essentiels ;
7. des quality gates explicites, avec critères d'acceptation ou de rejet ;
8. un déploiement local reproductible, de préférence avec Docker (optionnel, comme on a pas encore vu docker) ;
9. un monitoring minimal : qualité, erreurs, latence et, lorsque pertinent, coût ou consommation ;
10. une analyse des risques, limites, aspects éthiques et mesures de gouvernance.

Ouverture. Docker, Kubernetes, les architectures RAG multi-index, le routing intelligent entre index, LoRA et QLoRA ne sont pas des prérequis de ces mini-projets. Ils peuvent être proposés comme extensions facultatives. Une solution simple, reproductible et bien évaluée est préférable à une architecture complexe non maîtrisée.

3. Projet 1 – De l'étude de cas à l'industrialisation d'un assistant RAG

3.1 Contexte et objectif

Une organisation souhaite proposer un assistant capable de répondre aux questions des utilisateurs à partir d'une documentation de référence. L'assistant doit retrouver les passages pertinents, produire

une réponse fondée sur les documents et signaler clairement lorsqu'il ne dispose pas d'informations suffisantes.

Le projet reprend la logique métier du scénario de chatbot support client discuté au chapitre 2, mais vous devez construire vous-mêmes la solution technique. Si vous le souhaitez, conserver le domaine e-commerce ou choisir un autre domaine parmi les options proposées ci-dessous.

3.2 Choix du domaine

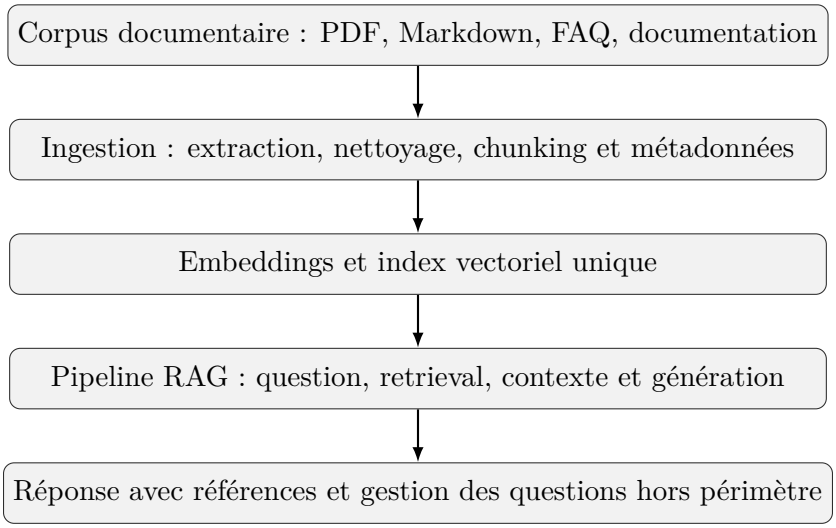
Les domaines suivants sont recommandés :

Domaine	**Exemples de questions**	**Documents possibles**
Support e-commerce	Livraison, retours, remboursement, garantie, suivi de commande.	FAQ, politique de retour, conditions de vente, guide de livraison.
Assistant universitaire	Inscriptions, examens, absences, stages, services aux étudiants.	Guide étudiant, règlement des examens, procédures de stage, FAQ.
Support informatique	Comptes, accès, réseau, logiciels, procédures de dépannage.	Base de connaissances, guides d'installation, procédures internes fictives.
Services administratifs	Démarches, pièces justificatives, délais, formulaires et procédures.	Guides publics, formulaires vierges, fiches pratiques.
etc	etc	etc

3.3 A réaliser

1. **Cadrage métier :** identifier les utilisateurs, les questions auxquelles l'assistant doit répondre, les questions hors périmètre et les conséquences d'une réponse incorrecte.
2. **Préparation documentaire :** inventorier les documents, vérifier leur qualité, extraire le texte, nettoyer les contenus et conserver leur provenance.
3. **Ingestion RAG :** définir le découpage en chunks, les métadonnées, le modèle d'embeddings et la stratégie d'indexation.
4. **Construction du pipeline :** implémenter retrieval, assemblage du contexte et génération de réponse. Le système doit pouvoir présenter les références documentaires utilisées.
5. **Gestion des limites :** prévoir un comportement lorsque le retrieval est insuffisant, lorsque la question est hors périmètre ou lorsque les documents sont contradictoires.
6. **Expérimentation :** comparer au moins deux configurations pertinentes, par exemple deux tailles de chunks, deux stratégies de découpage ou deux paramètres de retrieval.
7. **Évaluation :** mesurer la qualité de récupération et la qualité des réponses sur un jeu de questions annoté ou vérifié.
8. **Industrialisation :** versionner les documents et configurations, suivre les expérimentations, automatiser les tests et déployer le système.
9. **Gouvernance :** documenter les risques de réponses non fondées, de prompt injection documentaire, de fuite d'informations et de dépendance aux sources.

3.4 Architecture minimale attendue



L'architecture attendue est un **RAG monolithique avec un index vectoriel unique**. Plusieurs documents et formats peuvent être ingérés dans cet index commun. Vous n'avez pas à mettre en place plusieurs index ni un mécanisme de routing.

3.5 Évaluation et indicateurs

Vous devez au minimum étudier les dimensions suivantes :

- **Retrieval :** pertinence des passages retrouvés, Recall@k ou une mesure équivalente, et analyse qualitative des échecs ;
- **Génération :** exactitude, pertinence, fidélité au contexte documentaire et présence de références vérifiables ;
- **Robustesse :** questions hors corpus, formulations ambiguës, documents incomplets ou contradictoires ;
- **Performance :** latence, taux d'erreur et disponibilité observée pendant les essais ;
- **Coût :** estimation du coût par requête ou mesure des ressources consommées, selon le modèle utilisé.

Les seuils doivent être justifiés. Il est préférable de présenter les résultats détaillés et les limites plutôt que d'annoncer un score global sans expliquer sa méthode de calcul.

3.6 Livrables spécifiques

- corpus utilisé et inventaire des sources ;
- pipeline d'ingestion et de construction de l'index ;
- application RAG exécutable ;
- jeu de questions et résultats d'évaluation ;
- expériences MLflow et versions des configurations ;
- tests, workflow CI et quality gates ;
- rapport de risques et de gouvernance.

4. Projet 2 – RAG multi-sources : reproductibilité, évaluation et sécurité documentaire

4.1 Contexte et objectif

Une organisation dispose de plusieurs catégories de documents : guides PDF, pages Markdown, FAQ et documentation technique. Elle souhaite les exploiter dans un assistant documentaire tout en conservant la traçabilité des sources, la reproductibilité du pipeline et un niveau de sécurité adapté.

Le projet se concentre sur la qualité du cycle documentaire. Il ne demande pas de construire une architecture de recherche distribuée ou un système de routing entre plusieurs index.

4.2 Architecture imposée pour le périmètre obligatoire

Les différentes sources sont intégrées dans un **pipeline d'ingestion commun** et dans un **index vectoriel unique**. Chaque passage conserve des métadonnées utiles : identifiant du document, source, date ou version, type documentaire et, si nécessaire, niveau de sensibilité.

4.3 A réaliser

1. constituer un corpus comprenant plusieurs sources ou formats ;
2. définir une procédure d'extraction, de nettoyage, de découpage et d'indexation ;
3. conserver la provenance de chaque document et de chaque chunk ;
4. versionner le corpus, les paramètres de chunking, le modèle d'embeddings et la configuration de l'index ;
5. comparer au moins deux configurations de préparation ou de retrieval ;
6. évaluer la pertinence des passages récupérés et la qualité des réponses ;
7. mettre en place des tests de non-régression après modification du corpus ou de la configuration ;
8. documenter les risques de documents obsolètes, contradictoires, malveillants ou contenant des instructions non fiables ;
9. proposer un contrôle d'accès documentaire cohérent avec les métadonnées.

4.4 Sécurité documentaire

Le contrôle d'accès ne doit pas être appliqué uniquement après la récupération des meilleurs passages. Lorsqu'un document est réservé à une catégorie d'utilisateurs, le système doit filtrer les documents autorisés **avant ou pendant le retrieval**, selon les capacités de la base vectorielle utilisée. Il faut éviter qu'un passage non autorisé entre dans le contexte transmis au modèle, même si la réponse finale ne le cite pas.

Pour conserver un périmètre raisonnable, un contrôle simple peut suffire : documents publics et documents réservés à un groupe fictif d'utilisateurs. Vous devez expliquer les hypothèses retenues et tester au moins un cas d'accès autorisé et un cas d'accès refusé.

4.5 Livrables spécifiques

- corpus multi-sources et inventaire des métadonnées ;
- pipeline reproductible de préparation et d'indexation ;
- description des versions de corpus, embeddings et configurations ;
- protocole d'évaluation et résultats comparatifs ;

- tests de provenance, d'accès et de non-régression ;
- analyse des risques documentaires et mesures de protection.

Extension facultative uniquement. Si vous maîtrisez le périmètre de base, vous pouvez expérimenter plusieurs index ou un mécanisme de routing entre sources. Cette extension ne doit pas remplacer la réalisation, l'évaluation et la documentation du RAG monolithique demandé.

5. Projet 3 – Agent intelligent : orchestration d'outils, observabilité et gouvernance

5.1 Contexte et objectif

Une organisation souhaite automatiser certaines tâches d'assistance. Contrairement à un chatbot RAG classique qui répond principalement à partir de documents, l'agent peut sélectionner et appeler des outils pour obtenir des informations ou effectuer des opérations simulées.

Le projet porte sur l'orchestration contrôlée des outils, la traçabilité des interactions, l'évaluation du comportement et la gestion des risques.

5.2 Périmètre fonctionnel

L'agent doit disposer d'au moins trois outils. Les outils peuvent être simulés afin d'éviter les intégrations externes et de maintenir le projet dans une durée raisonnable.

Exemples :

- recherche dans une base documentaire ou un petit corpus RAG ;
- consultation du statut d'une commande fictive ;
- recherche d'un ticket de support fictif ;
- calcul d'un montant de remboursement selon des règles prédéfinies ;
- création d'une demande d'escalade vers un opérateur humain.

Choisissez un scénario cohérent et décrit les entrées, sorties, erreurs possibles et autorisations de chaque outil.

5.3 Travail demandé

1. définir le scénario, les objectifs de l'agent et les tâches qu'il ne doit pas réaliser ;
2. implémenter l'orchestration et au moins trois outils ;
3. valider les arguments avant chaque appel d'outil ;
4. gérer les erreurs, les délais d'attente, les réponses invalides et les limites d'itération ;
5. tracer les événements opérationnels de chaque interaction ;
6. constituer un jeu de scénarios nominaux, d'erreurs et de tentatives d'utilisation hors périmètre ;
7. évaluer le succès des tâches, la sélection des outils, les erreurs et les escalades ;
8. automatiser les tests et intégrer des quality gates ;
9. documenter les risques, les permissions et les conditions de supervision humaine.

5.4 Traçabilité et observabilité

Les traces doivent permettre de reconstruire le déroulement opérationnel d'une interaction. Elles peuvent notamment contenir :

- identifiant de requête et horodatage ;
- outil sélectionné et motif opérationnel de sélection, sous forme de catégorie ou d'événement ;
- arguments transmis à l'outil, après suppression ou masquage des données sensibles ;
- résultat, statut, durée et éventuelle erreur ;
- nombre d'appels, nombre d'itérations et décision d'escalade ;
- réponse finale, latence et coût estimé lorsque disponible.

Limite importante. Il n'est pas demandé de collecter ou d'enregistrer le raisonnement interne privé du modèle. La traçabilité porte sur les événements observables du système : appels d'outils, paramètres, résultats, validations, erreurs, durées et décisions opérationnelles. Les journaux doivent être conçus pour limiter l'exposition de données personnelles ou confidentielles.

5.5 Évaluation et sécurité

Vous devez évaluer au minimum :

- le taux de réussite des tâches ;
- la pertinence de la sélection des outils ;
- le taux d'appels invalides ou échoués ;
- la latence et le nombre moyen d'appels par tâche ;
- le comportement face à une demande ambiguë ou non autorisée ;
- la capacité à refuser une action risquée ou à demander une validation humaine.

Les opérations ayant un effet métier simulé (par exemple remboursement ou modification d'une commande) doivent être soumises à des règles explicites et, si nécessaire, à une confirmation humaine. Les outils ne doivent pas disposer de permissions plus larges que celles nécessaires au scénario.

5.6 Livrables spécifiques

- description du scénario et des outils ;
- agent exécutable et outils simulés ;
- traces opérationnelles documentées ;
- jeu de scénarios et résultats d'évaluation ;
- tests des outils, tests d'intégration et tests de bout en bout ;
- règles de sécurité, d'escalade et de supervision humaine.

6. Projet 4 - Industrialisation d'un pipeline d'apprentissage automatique classique

6.1 Contexte et objectif

Ce projet vise à appliquer les pratiques MLOps à un système d'apprentissage automatique qui ne repose ni sur un LLM, ni sur un RAG, ni sur un agent. Il permet de vérifier que les compétences du cours sont transférables au cycle de vie ML classique.

6.2 Domaines possibles

- prédiction du désabonnement (churn) ;
- prévision de la demande ou de la consommation énergétique ;
- maintenance prédictive ;
- détection d'anomalies ;
- classification de défauts ou de qualité ;
- prédiction d'un indicateur à partir d'un jeu de données public.

Les données doivent être accessibles légalement, documentées et adaptées à une expérimentation reproductible. Vous devez vérifier les risques de fuite de données, de biais et de mauvaise interprétation des métriques.

6.3 Travail demandé

1. formuler le problème et définir une métrique principale adaptée ;
2. explorer les données et documenter les choix de préparation ;
3. construire un pipeline de prétraitement et d'entraînement reproductible ;
4. comparer au moins deux modèles ou configurations ;
5. enregistrer les paramètres, métriques, artefacts et versions avec MLflow ;
6. versionner les données et le code avec Git et DVC ou un outil équivalent ;
7. enregistrer une version candidate dans un registre de modèles ou mettre en place un mécanisme documenté de gestion des versions ;
8. exposer le modèle via une API simple et le conteneuriser (optionnel) ;
9. automatiser les tests et les quality gates ;
10. simuler un suivi en production : qualité des entrées, erreurs, latence et, lorsque possible, dérive des données ou des prédictions.

6.4 Évaluation et gouvernance

Le rapport doit préciser le protocole de séparation des données, les risques de fuite, les limites de généralisation et les conséquences possibles d'une erreur. Vous devez distinguer les métriques hors ligne de la qualité réellement observable après déploiement.

6.5 Livrables spécifiques

- pipeline de données et d'entraînement ;
- expériences et artefacts MLflow ;
- données et code versionnés ;
- modèle enregistré et API déployable (optionnel) ;
- tests, CI et quality gates ;
- indicateurs de monitoring et analyse des limites.

7. Extensions facultatives

Les extensions suivantes ne sont pas nécessaires pour obtenir une bonne évaluation. Elles ne doivent être entreprises qu'après validation du périmètre obligatoire.

Extension	**Conditions et précautions**
RAG multi-index et routing	Comparer une architecture à plusieurs index à l'index unique du projet 2. Le routing intelligent est une notion avancée et ne constitue pas une exigence du cours.
LoRA / QLoRA	Présenter ou expérimenter une adaptation légère d'un modèle. Il faut distinguer l'adaptation du modèle de l'accès à des connaissances externes assuré par le RAG. Aucun fine-tuning n'est exigé.
Comparaison RAG et fine-tuning	Comparer les objectifs, les coûts, les limites et les résultats sur un cas restreint, sans supposer qu'une approche est systématiquement meilleure.
Observabilité avancée	Ajouter un tableau de bord, des alertes ou une analyse plus détaillée des traces, à condition que les indicateurs de base soient déjà disponibles.
Déploiement orchestré	Un déploiement sur Kubernetes peut être proposé si vous disposez du temps et des compétences nécessaires. Il n'est pas obligatoire.

8. Socle de qualité : tests et quality gates

Vous devez définir des contrôles automatiques qui empêchent la promotion d'une version manifestement dégradée. Les quality gates doivent être liés au risque et au type de système.

Niveau	**Exemples de contrôles**	**Résultat attendu**
Code	Tests unitaires, lint, validation de configuration, gestion des exceptions.	Échec du pipeline si les tests essentiels échouent.
Données	Schéma, valeurs manquantes, doublons, format, provenance, version.	Détection des données invalides ou inattendues.
Modèle / RAG / agent	Métriques métier, qualité retrieval, fidélité, tests de scénarios, comportement hors périmètre.	Acceptation uniquement si les critères définis sont satisfaits.
Sécurité	Dépendances, secrets, images de conteneurs, entrées non fiables, permissions.	Aucune vulnérabilité critique non traitée dans le périmètre contrôlé.
Déploiement	Démarrage, endpoint de santé, test d'intégration, configuration.	Version déployable et vérifiée.

Les seuils numériques ne doivent pas être choisis arbitrairement. Vous devez expliquer leur origine, leur pertinence et les limites de leur mesure. Un seuil peut être défini à partir d'une baseline, d'une exigence métier ou d'un compromis documenté.

9. Monitoring minimal attendu

Le monitoring doit être adapté au système développé. Un tableau de bord très simple est acceptable si les métriques sont pertinentes, correctement définies et interprétées.

Dimension	**Exemples d'indicateurs**	**Applicabilité**
Disponibilité et erreurs	Taux d'erreur, requêtes échouées, disponibilité de l'API.	Tous les projets.
Performance	Latence moyenne et percentile élevé (par exemple P95).	Tous les projets.
Qualité	Métrique ML, qualité des réponses, qualité retrieval ou taux de réussite des tâches.	Selon le projet.
Coût et ressources	Coût estimé par requête, temps CPU/GPU, mémoire ou volume de tokens.	Selon les outils utilisés.
Données et dérive	Évolution des distributions, données manquantes, dérive des entrées ou changements du corpus.	Selon le projet.

10. Livrables communs

Vous devez remettre les éléments suivants :

1. **Dépôt Git (GitHub) :** code, configurations, tests, workflow CI et documentation ;
2. **README :** présentation, architecture, installation, exécution et démonstration ;
3. **Rapport technique de 8 à 12 pages :** problème, architecture, données, choix techniques, expériences, résultats, limites, risques et gouvernance ;
4. **Dossier de reproductibilité :** versions, commandes, paramètres, données ou identifiants de versions, artefacts et procédure de reconstruction ;
5. **Résultats d'évaluation :** métriques, tableaux ou graphiques, analyse des erreurs et justification des quality gates ;
6. **Démonstration de 2 minutes (en vidéo pour évitez les surprises, vous pouvez faire en temps rél si vous le souhaitez)! :** exécution du système, présentation de la chaîne MLOps et discussion d'un cas d'échec ou d'une limite.

Les secrets, clés API, mots de passe et données personnelles ne doivent jamais être déposés dans le dépôt Git. Vous devez fournir un fichier d'exemple de configuration sans valeur secrète.

11. Conseils Pratiques

Avant de commencer le développement, vous devez répondre clairement aux questions suivantes :

1. Quel problème précis le système cherche-t-il à résoudre ?
2. Quelles données ou quels documents sont utilisés, et avec quels droits ?
3. Quel est le résultat attendu et comment sera-t-il évalué ?
4. Quelles versions du code, des données, des modèles et des configurations doivent être conservées ?

5. Quels tests empêchent une régression ?
6. Quels événements et indicateurs seront observés après le déploiement ?
7. Quels sont les risques principaux et quelles mesures permettent de les réduire ?

NB. Le but du mini-projet n'est pas de construire le système le plus sophistiqué possible. Il est de démontrer qu'une équipe sait concevoir un système d'IA utile, mesurer ses performances, expliquer ses limites, reconstruire ses expériences et organiser son passage vers un fonctionnement contrôlé.